

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
from statsmodels.tsa.arima.model import ARIMA
import itertools
from numpy.polynomial.polynomial import polyfit

%pip install openpyxl
%pip install mlxtend
%pip install statsmodels
```

Requirement already satisfied: openpyxl in c:\users\rayantech\programs\python\python312\lib\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\rayantech\programs\python\python312\lib\site-packages (from openpyxl) (2.0.0)
Note: you may need to restart the kernel to use updated packages.

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\argparse-1.4.0-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\astor-0.8.1-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\pyggy-2.0-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

[notice] A new release of pip is available: 25.0.1 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

Requirement already satisfied: mlxtend in c:\users\rayantech\programs\python\python312\lib\site-packages (0.23.4)

Requirement already satisfied: scipy>=1.2.1 in c:\users\rayantech\programs\python\python312\lib\site-packages (from mlxtend) (1.16.3)

Requirement already satisfied: numpy>=1.16.2 in c:\users\rayantech\programs\python\python312\lib\site-packages (from mlxtend) (2.3.4)

Requirement already satisfied: pandas>=0.24.2 in c:\users\rayantech\programs\python\python312\lib\site-packages (from mlxtend) (2.3.3)

Requirement already satisfied: scikit-learn>=1.3.1 in c:\users\rayantech\programs\python\python312\lib\site-packages (from mlxtend) (1.7.2)

Requirement already satisfied: matplotlib>=3.0.0 in c:\users\rayantech\programs\python\python312\lib\site-packages (from mlxtend) (3.10.7)

Requirement already satisfied: joblib>=0.13.2 in c:\users\rayantech\programs\python\python312\lib\site-packages (from mlxtend) (1.5.2)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\rayantech\programs\python\python312\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (1.3.3)

Requirement already satisfied: cycler>=0.10 in c:\users\rayantech\programs\python\python312\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\rayantech\programs\python\python312\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\rayantech\programs\python\python312\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (1.4.9)

Requirement already satisfied: packaging>=20.0 in c:\users\rayantech\appdata\roaming\python\python312\site-packages (from matplotlib>=3.0.0->mlxtend) (25.0)

Requirement already satisfied: pillow>=8 in c:\users\rayantech\programs\python\python312\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (12.0.0)

Requirement already satisfied: pyparsing>=3 in c:\users\rayantech\programs\python\python312\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\rayantech\appdata\roaming\python\python312\site-packages (from matplotlib>=3.0.0->mlxtend) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\rayantech\programs\python\python312\lib\site-packages (from pandas>=0.24.2->mlxtend) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\rayantech\programs\python\python312\lib\site-packages (from pandas>=0.24.2->mlxtend) (2025.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\rayantech\programs\python\python312\lib\site-packages (from scikit-learn>=1.3.1->mlxtend) (3.6.0)

Requirement already satisfied: six>=1.5 in c:\users\rayantech\appdata\roaming\python\python312\site-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.17.0)

Note: you may need to restart the kernel to use updated packages.

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\argparse-1.4.0-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\astor-0.8.1-py3.12.egg is deprecated. pip

25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\pyggi-2.0-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

[notice] A new release of pip is available: 25.0.1 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

Requirement already satisfied: statsmodels in c:\users\rayantech\programs\python\python312\lib\site-packages (0.14.5)

Requirement already satisfied: numpy<3,>=1.22.3 in c:\users\rayantech\programs\python\python312\lib\site-packages (from statsmodels) (2.3.4)

Requirement already satisfied: scipy!=1.9.2,>=1.8 in c:\users\rayantech\programs\python\python312\lib\site-packages (from statsmodels) (1.16.3)

Requirement already satisfied: pandas!=2.1.0,>=1.4 in c:\users\rayantech\programs\python\python312\lib\site-packages (from statsmodels) (2.3.3)

Requirement already satisfied: patsy>=0.5.6 in c:\users\rayantech\programs\python\python312\lib\site-packages (from statsmodels) (1.0.2)

Requirement already satisfied: packaging>=21.3 in c:\users\rayantech\appdata\roaming\python\python312\site-packages (from statsmodels) (25.0)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\rayantech\appdata\roaming\python\python312\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\rayantech\programs\python\python312\lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\rayantech\programs\python\python312\lib\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\rayantech\appdata\roaming\python\python312\site-packages (from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)

Note: you may need to restart the kernel to use updated packages.

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\argparse-1.4.0-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\astor-0.8.1-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at

<https://github.com/pypa/pip/issues/12330>

DEPRECATION: Loading egg at c:\users\rayantech\programs\python\python312\lib\site-packages\pyggi-2.0-py3.12.egg is deprecated. pip 25.1 will enforce this behaviour change. A possible replacement is to use pip for package installation. Discussion can be found at <https://github.com/pypa/pip/issues/12330>

[notice] A new release of pip is available: 25.0.1 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

```
df = pd.read_excel("Online Retail.xlsx")
print(df.head)
print(df.shape)
print(df.info())
```

```
<bound method NDFrame.head of
Description  Quantity  \
0          536365    85123A  WHITE HANGING HEART T-LIGHT HOLDER
6
1          536365    71053                WHITE METAL LANTERN
6
2          536365    84406B    CREAM CUPID HEARTS COAT HANGER
8
3          536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE
6
4          536365    84029E    RED WOOLLY HOTTIE WHITE HEART.
6
...          ...          ...          ...
...
541904    581587    22613                PACK OF 20 SPACEBOY NAPKINS
12
541905    581587    22899    CHILDREN'S APRON DOLLY GIRL
6
541906    581587    23254    CHILDRENS CUTLERY DOLLY GIRL
4
541907    581587    23255    CHILDRENS CUTLERY CIRCUS PARADE
4
541908    581587    22138    BAKING SET 9 PIECE RETROSPOT
3
```

	InvoiceDate	UnitPrice	CustomerID	Country
0	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
...	...	...	...	...
541904	2011-12-09 12:50:00	0.85	12680.0	France
541905	2011-12-09 12:50:00	2.10	12680.0	France
541906	2011-12-09 12:50:00	4.15	12680.0	France

541907	2011-12-09	12:50:00	4.15	12680.0	France
541908	2011-12-09	12:50:00	4.95	12680.0	France

```
[541909 rows x 8 columns]>
(541909, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode       541909 non-null object
2   Description      540455 non-null object
3   Quantity        541909 non-null int64
4   InvoiceDate      541909 non-null datetime64[ns]
5   UnitPrice       541909 non-null float64
6   CustomerID      406829 non-null float64
7   Country         541909 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB
None
```

Handle Missing Values

```
print("Before removing missing values:\n", df.isna().sum())

Before removing missing values:
InvoiceNo      0
StockCode      0
Description    1454
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID    135080
Country        0
dtype: int64

df = df[df["Description"].notna() & (df["Description"] != "")]
# Define non-product stock codes to exclude
non_product_stockcodes = [
    "POST", # Postage
    "DOT", # DOTCOM POSTAGE
    "D", # Discount
    "M", # Manual
    "m", # Manual (lowercase)
    "C2", # Carriage
    "AMAZONFEE", # Amazon marketplace fee
    "BANK CHARGES", # Bank charges
    "CRUK", # Charity commission (Cancer Research UK)
```

```

        "S", # Samples
        "B", # Bad debt adjustment
        "PADS", # Pads (service item)
    ]

df["StockCode"] = df["StockCode"].astype(str).str.strip().str.upper()

rows_before_stockcode = len(df)
non_product_mask = df["StockCode"].isin(
    [code.upper() for code in non_product_stockcodes]
)
non_product_counts = df[non_product_mask]["StockCode"].value_counts()
if len(non_product_counts) > 0:
    print("Non-product stock codes found:")
    for code, count in non_product_counts.items():
        print(f" - {code}: {count} records")

df = df[~non_product_mask]
print(f"Removed {rows_before_stockcode - len(df)} non-product stock
code records")

print("\n--- 3. Description Standardization ---")
df["Description"] =
df["Description"].astype(str).str.strip().str.lower()
print("All descriptions converted to lowercase and whitespace
stripped")

anomaly_description_patterns = [
    "adjust", # Adjustments
    "bad debt", # Bad debt write-offs
    "damages", # Damaged goods
    "lost", # Lost inventory
    "incorrect",
    "thrown away", # Discarded items
    "crushed", # Damaged in transit
    "wet", # Water damage
    "rusty", # Rust damage
    "mouldy", # Mold damage
    "smashed", # Broken items
    "broken", # Broken items
    "amazon fee", # Amazon fees
    "bank charges", # Bank fees
    "carriage", # Carriage/delivery charges
    "dotcom", # Dotcom adjustments
    "given away", # Free items
    "mailout", # Mailing costs
    "manual", # Manual adjustments
    "cruk", # Charity commission
    "samples", # Sample items
    "?", # Unknown/unclear items (question marks)

```

```

    "allocate stock", # Stock allocation entries
    "check", # Check/verification entries
    "incorrect", # Incorrect entries
    "wrong", # Wrong entries
    "test", # Test entries
    "mixed up", # Mixed up items
    "sold as set", # Adjustment for set items
    "counted", # Counting adjustments
    "show sample", # Show samples
    "had been put aside", # Put aside items
    "amazon adjust", # Amazon adjustments
    "found", # Found items (inventory adjustments)
]

rows_before_desc_patterns = len(df)
anomaly_desc_mask = pd.Series([False] * len(df), index=df.index)

print("Anomalous description patterns found:")
for pattern in anomaly_description_patterns:
    pattern_mask = df["Description"].str.contains(pattern, na=False,
    regex=False)
    pattern_count = pattern_mask.sum()
    if pattern_count > 0:
        print(f" - '{pattern}': {pattern_count} records")
        anomaly_desc_mask = anomaly_desc_mask | pattern_mask

df = df[~anomaly_desc_mask]
print(
    f"Removed {rows_before_desc_patterns - len(df)} records with
    anomalous descriptions"
)
print("After removing missing values:\n", df.isna().sum())
print("After removing missing values:\n", df.isna().sum())

```

Non-product stock codes found:

- POST: 1252 records
- DOT: 709 records
- M: 572 records
- C2: 143 records
- D: 77 records
- S: 63 records
- BANK CHARGES: 37 records
- AMAZONFEE: 34 records
- CRUK: 16 records
- PADS: 4 records
- B: 3 records

Removed 2910 non-product stock code records

--- 3. Description Standardization ---

All descriptions converted to lowercase and whitespace stripped

Anomalous description patterns found:

- 'adjust': 31 records
- 'damages': 61 records
- 'lost': 6 records
- 'incorrect': 7 records
- 'thrown away': 20 records
- 'crushed': 5 records
- 'wet': 17 records
- 'rusty': 8 records
- 'mouldy': 4 records
- 'smashed': 4 records
- 'broken': 1 records
- 'carriage': 178 records
- 'dotcom': 83 records
- 'given away': 1 records
- 'mailout': 4 records
- 'samples': 4 records
- '?': 82 records
- 'allocate stock': 1 records
- 'check': 418 records
- 'incorrect': 7 records
- 'wrong': 19 records
- 'test': 3 records
- 'mixed up': 2 records
- 'sold as set': 26 records
- 'counted': 2 records
- 'show sample': 1 records
- 'had been put aside': 6 records
- 'amazon adjust': 2 records
- 'found': 38 records

Removed 975 records with anomalous descriptions

After removing missing values:

InvoiceNo	0
StockCode	0
Description	0
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	131963
Country	0

dtype: int64

After removing missing values:

InvoiceNo	0
StockCode	0
Description	0
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	131963


```
Country          0
dtype: int64
```

Remove invalid records

```
bad_cnt = df[(df["Quantity"] <= 0)].shape[0]
print("Rows with zero or negative quantity BEFORE:", bad_cnt)

df_clean = df[df["Quantity"] > 0]
after_cnt = df_clean[(df_clean["Quantity"] <= 0)].shape[0]
print("Rows with zero or negative quantity AFTER:", after_cnt)
df = df_clean

Rows with zero or negative quantity BEFORE: 8796
Rows with zero or negative quantity AFTER: 0

bad_cnt = df[(df["UnitPrice"] <= 0)].shape[0]
print("Rows with zero or negative UnitPrice BEFORE:", bad_cnt)

df_clean = df[df["UnitPrice"] > 0]
after_cnt = df_clean[(df_clean["UnitPrice"] <= 0)].shape[0]
print("Rows with zero or negative quantity AFTER:", after_cnt)
df = df_clean

Rows with zero or negative UnitPrice BEFORE: 448
Rows with zero or negative quantity AFTER: 0
```

Data type conversion

```
print("BEFORE: ", df["InvoiceDate"].dtype)
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
print("AFTER: ", df["InvoiceDate"].dtype)

BEFORE:  datetime64[ns]
AFTER:  datetime64[ns]
```

Feature engineering

```
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
print(df.columns, df["Quantity"].head, df["UnitPrice"].head,
df["TotalAmount"].head)

Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity',
      'InvoiceDate',
      'UnitPrice', 'CustomerID', 'Country', 'TotalAmount'],
      dtype='object') <bound method NDFrame.head of 0      6
1      6
2      8
3      6
4      6
```

```

541904    12
541905     6
541906     4
541907     4
541908     3
Name: Quantity, Length: 527326, dtype: int64> <bound method
NDFrame.head of 0      2.55
1      3.39
2      2.75
3      3.39
4      3.39
...
541904    0.85
541905    2.10
541906    4.15
541907    4.15
541908    4.95
Name: UnitPrice, Length: 527326, dtype: float64> <bound method
NDFrame.head of 0      15.30
1      20.34
2      22.00
3      20.34
4      20.34
...
541904    10.20
541905    12.60
541906    16.60
541907    16.60
541908    14.85
Name: TotalAmount, Length: 527326, dtype: float64>

df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]
print(df.columns)

Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity',
      'InvoiceDate',
      'UnitPrice', 'CustomerID', 'Country', 'TotalAmount'],
      dtype='object')

original_count = df.shape[0]
# df_grouped = df.groupby(["InvoiceNo", "Description"],
# as_index=False)["Quantity"].sum()
df["Quantity_sum"] = df.groupby(["InvoiceNo", "Description"])
["Quantity"].transform("sum")
df_grouped = df.drop_duplicates(subset=["InvoiceNo", "Description"])

new_count = df_grouped.shape[0]
print("Before:", original_count)
print("After:", new_count)

```

```

print(df_grouped.columns, df.columns)

Before: 527326
After: 516565
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity',
      'InvoiceDate',
      'UnitPrice', 'CustomerID', 'Country', 'TotalAmount',
      'Quantity_sum'],
      dtype='object') Index(['InvoiceNo', 'StockCode', 'Description',
      'Quantity', 'InvoiceDate',
      'UnitPrice', 'CustomerID', 'Country', 'TotalAmount',
      'Quantity_sum'],
      dtype='object')

product_counts = df_grouped["Description"].value_counts()
frequent_products = product_counts[product_counts >= 10].index
df_filtered =
df_grouped[df_grouped["Description"].isin(frequent_products)]
print(df_filtered["Description"].value_counts().min())

df_grouped = df_filtered

10

```

Preprocessing for Association Rule Mining

```

baskets = df_grouped.groupby("InvoiceNo")
["Description"].apply(list).reset_index()
print(baskets.head)

```

```

<bound method NDFrame.head of      InvoiceNo
Description
0      536365  [white hanging heart t-light holder, white met...
1      536366  [hand warmer union jack, hand warmer red polka...
2      536367  [assorted colour bird ornament, poppy's playho...
3      536368  [jam making set with jars, red coat rack paris...
4      536369  [bath building block word]
...
19731    581583  [lunch bag red retrospot, 6 chocolate love hea...
19732    581584  [red flock love heart photo frame, 6 chocolate...
19733    581585  [black tea towel classic design, assorted bott...
19734    581586  [large cake stand hanging strawberry, set of 3...
19735    581587  [circus parade lunch box, plasters in tin circ...

[19736 rows x 2 columns]>

```

Preprocessing for Time Series Analysis

```

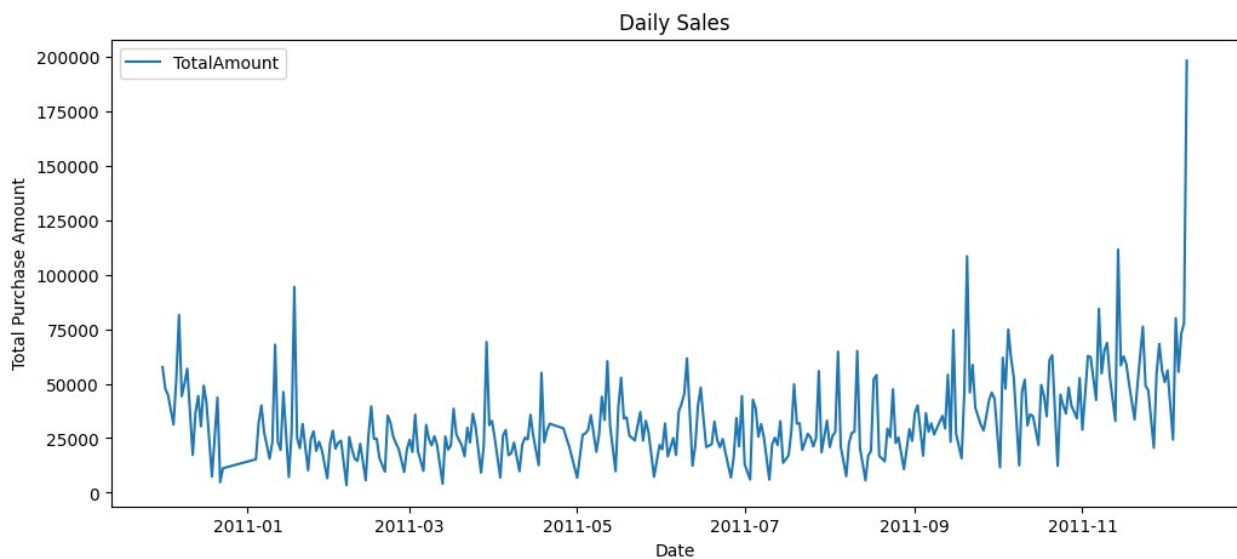
print(df.columns)

df["InvoiceDay"] = df["InvoiceDate"].dt.date
daily_sales = df.groupby("InvoiceDay")
["TotalAmount"].sum().reset_index()
daily_sales.columns = ["Date", "TotalAmount"]
daily_sales = daily_sales.sort_values("Date")

daily_sales.plot(x="Date", y="TotalAmount", kind="line",
figsize=(12,5))
plt.ylabel("Total Purchase Amount")
plt.title("Daily Sales")
plt.show()

Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity',
      'InvoiceDate',
      'UnitPrice', 'CustomerID', 'Country', 'TotalAmount',
      'Quantity_sum'],
      dtype='object')

```



Exploratory Data Analysis (EDA)

General Dataset Exploration

```

print(df.describe(), df.columns)

price_95 = df["UnitPrice"].quantile(0.95)
quantity_95 = df["Quantity"].quantile(0.95)
df = df[(df["UnitPrice"] <= price_95) & (df["Quantity"] <=
quantity_95)]
df_filtered = df[(df["Quantity"] > 0) &
                  (df["UnitPrice"] > 0) &

```

```

(df["UnitPrice"] <= price_95) &
(df["Quantity"] <= quantity_95)]
plt.figure(figsize=(8, 4))
plt.hist(df_filtered["UnitPrice"], bins=50, alpha=0.5, color='red')
plt.xlabel("UnitPrice")
plt.ylabel("Frequency (log scale)")
plt.title("Distribution of UnitPrice (positive values, log scale)")
plt.grid(alpha=0.3)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.hist(df_filtered["Quantity"], bins=50, alpha=0.5, color='blue')
plt.xlabel("Quantity")
plt.ylabel("Frequency (log scale)")
plt.title("Distribution of Quantity (positive values, log scale)")
plt.grid(alpha=0.3)
plt.show()

```

```

top_products = df["Description"].value_counts().head(15)
print(top_products)

```

```

plt.figure(figsize=(12,6))
sns.barplot(x=top_products.values, y=top_products.index,
palette="viridis")
plt.title("Top 15 Most Frequently Purchased Products")
plt.xlabel("Number of Purchases")
plt.ylabel("Product")
plt.show()

```

```

df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
country_sales = df.groupby("Country")
["TotalAmount"].sum().sort_values(ascending=False).head(15)
plt.figure(figsize=(12,6))
sns.barplot(x=country_sales.values, y=country_sales.index,
palette="coolwarm")
plt.title("Top 15 Countries by Total Sales")
plt.xlabel("Total Sales")
plt.ylabel("Country")
plt.show()

```

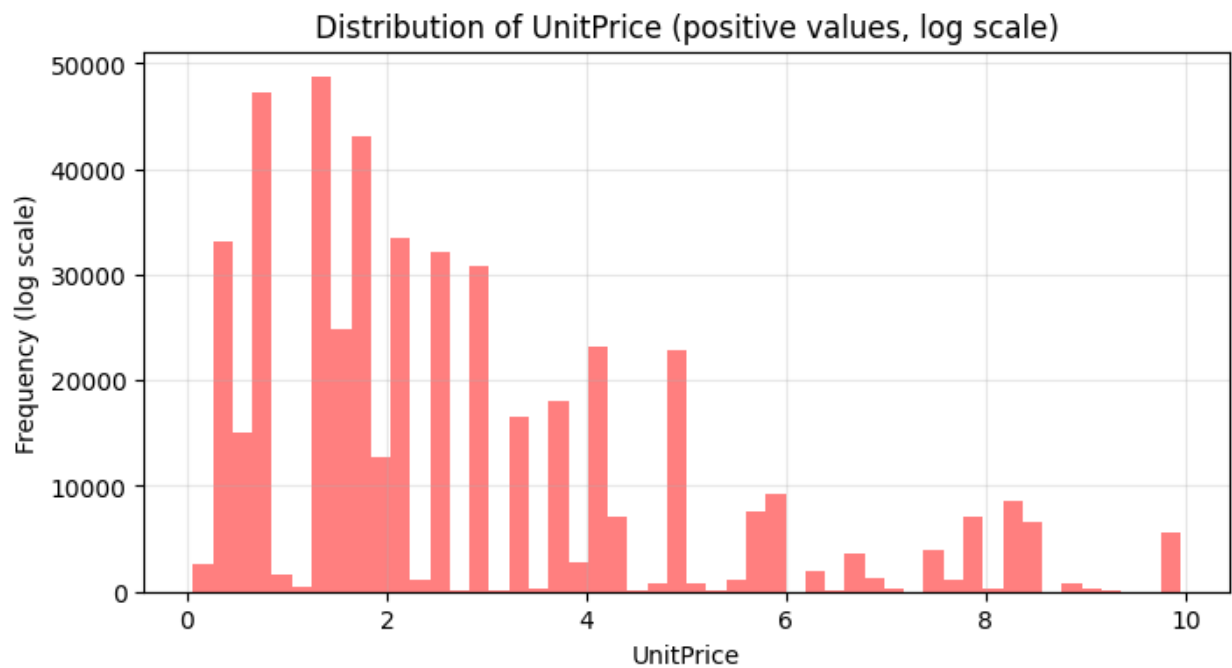
	Quantity	InvoiceDate	UnitPrice
count	527326.000000	527326	527326.000000
mean	10.574472	2011-07-04 21:46:56.296180992	3.261314
min	1.000000	2010-12-01 08:26:00	0.040000
25%	1.000000	2011-03-28 12:23:00	1.250000
50%	3.000000	2011-07-20 13:30:00	2.080000
75%	11.000000	2011-10-19 13:58:00	4.130000
max	80995.000000	2011-12-09 12:50:00	649.500000
std	155.898030	NaN	4.374171

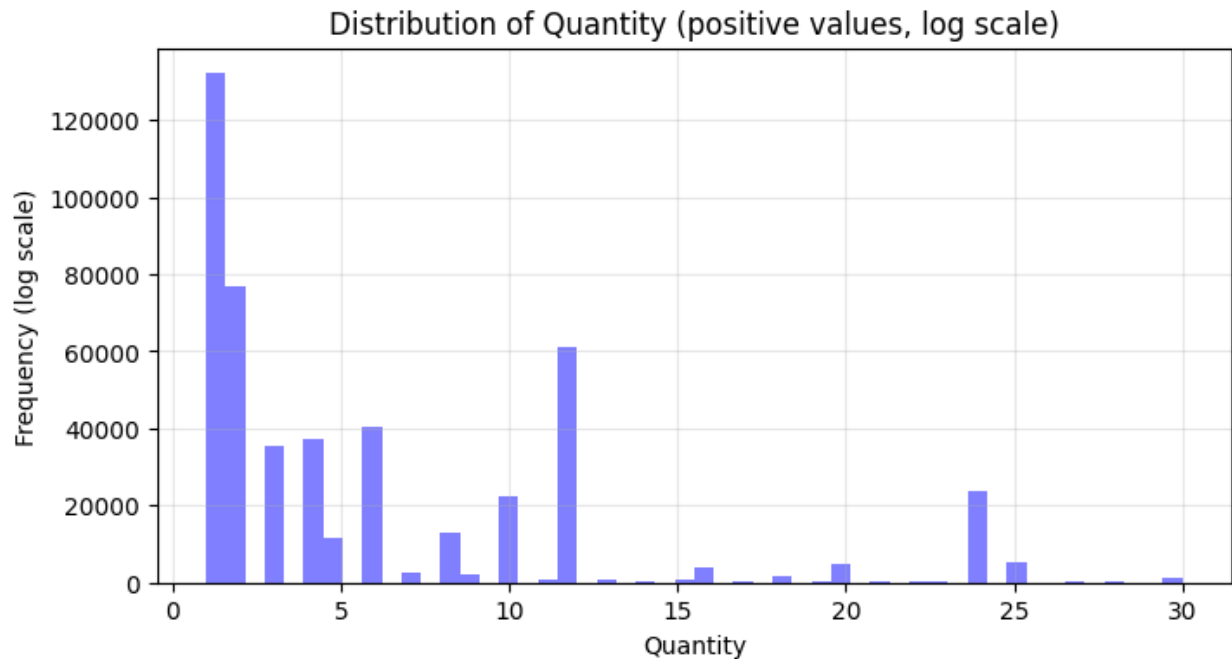
	CustomerID	TotalAmount	Quantity_sum
count	396042.000000	527326.000000	527326.000000
mean	15301.627916	19.466690	10.742266
min	12346.000000	0.060000	1.000000
25%	13975.000000	3.750000	1.000000
50%	15159.000000	9.900000	4.000000
75%	16803.000000	17.400000	12.000000
max	18287.000000	168469.600000	80995.000000
std	1710.050083	268.715351	156.010202

```

Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity',
      'InvoiceDate',
      'UnitPrice', 'CustomerID', 'Country', 'TotalAmount',
      'Quantity_sum',
      'InvoiceDay'],
      dtype='object')

```





```

Description
white hanging heart t-light holder      1972
jumbo bag red retrospot                  1870
lunch bag red retrospot                  1503
party bunting                           1347
lunch bag black skull.                   1289
pack of 72 retrospot cake cases          1244
assorted colour bird ornament            1238
natural slate heart chalkboard            1226
lunch bag spaceboy design                 1152
set of 3 cake tins pantry design          1151
jumbo bag pink polkadot                   1141
lunch bag cars blue                       1140
jumbo shopper vintage red paisley         1134
jumbo storage bag suki                    1128
heart of wicker small                     1127
Name: count, dtype: int64

```

```

C:\Users\RayanTech\AppData\Local\Temp\ipykernel_4264\3359277287.py:30:
FutureWarning:

```

```

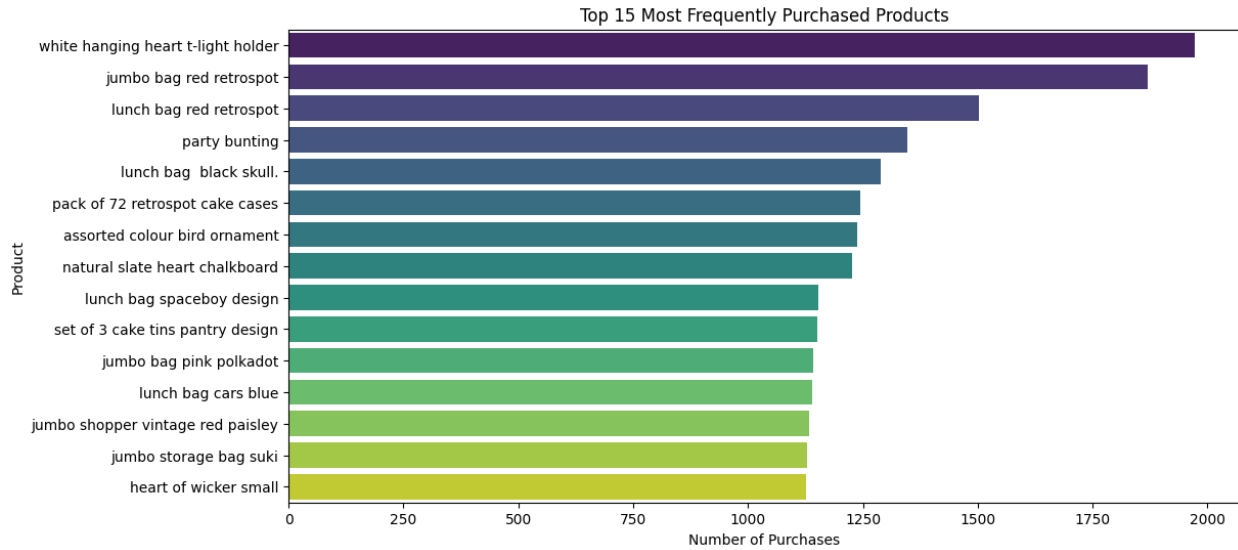
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `y` variable to `hue` and set
`legend=False` for the same effect.

```

```

sns.barplot(x=top_products.values, y=top_products.index,
palette="viridis")

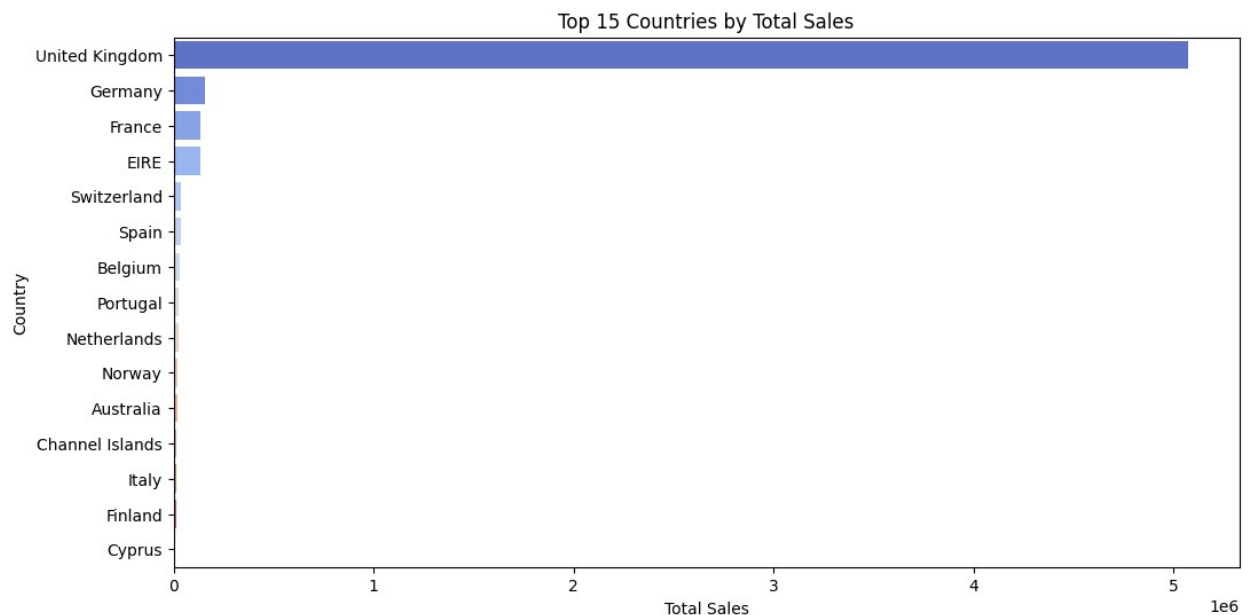
```



C:\Users\RayanTech\AppData\Local\Temp\ipykernel_4264\3359277287.py:39: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=country_sales.values, y=country_sales.index,
palette="coolwarm")
```



EDA for Association Rules


```

print(baskets)
baskets["NumProducts"] = baskets["Description"].apply(len)
print(baskets.head())

plt.figure(figsize=(10,5))
plt.hist(baskets["NumProducts"], bins=30, range=(0, 200),
color="skyblue", alpha=0.5)
plt.xlabel("Number of Products per Transaction")
plt.ylabel("Number of Transactions")
plt.title("Distribution of Number of Products per Transaction")
plt.grid(alpha=0.3)
plt.show()

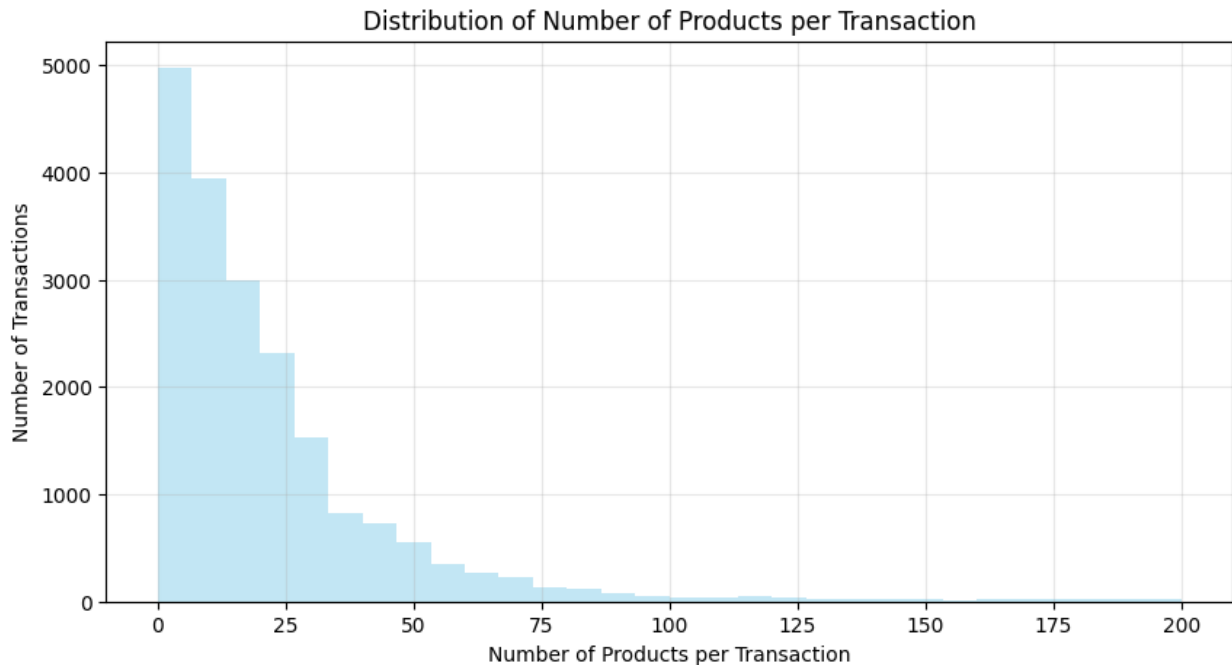
print("Mean products per transaction:", baskets["NumProducts"].mean())
print("Median products per transaction:",
baskets["NumProducts"].median())
print("Max products per transaction:", baskets["NumProducts"].max())

```

	InvoiceNo	Description
0	536365	[white hanging heart t-light holder, white met...
1	536366	[hand warmer union jack, hand warmer red polka...
2	536367	[assorted colour bird ornament, poppy's playho...
3	536368	[jam making set with jars, red coat rack paris...
4	536369	[bath building block word]
...	...	...
19731	581583	[lunch bag red retrospot, 6 chocolate love hea...
19732	581584	[red flock love heart photo frame, 6 chocolate...
19733	581585	[black tea towel classic design, assorted bott...
19734	581586	[large cake stand hanging strawberry, set of 3...
19735	581587	[circus parade lunch box, plasters in tin circ...

[19736 rows x 2 columns]

	InvoiceNo	Description
NumProducts		
0	536365	[white hanging heart t-light holder, white met...
7		
1	536366	[hand warmer union jack, hand warmer red polka...
2		
2	536367	[assorted colour bird ornament, poppy's playho...
11		
3	536368	[jam making set with jars, red coat rack paris...
4		
4	536369	[bath building block word]
1		



Mean products per transaction: 26.013275233076612

Median products per transaction: 15.0

Max products per transaction: 1086

```
print(baskets)
all_items = baskets["Description"].explode()
item_counts = all_items.value_counts()
top_items = item_counts.head(15)
top_products = df["Description"].value_counts().head(15)
```

```
all_items = baskets["Description"].explode()
item_counts = all_items.value_counts()
top_items = item_counts.head(15)

plt.figure(figsize=(12,6))
sns.barplot(x=top_items.values, y=top_items.index, palette="viridis")
plt.title("Top 15 Most Frequently Purchased Items")
plt.xlabel("Number of Purchases")
plt.ylabel("Product")
plt.show()
```

	InvoiceNo	Description \
0	536365	[white hanging heart t-light holder, white met...
1	536366	[hand warmer union jack, hand warmer red polka...
2	536367	[assorted colour bird ornament, poppy's playho...
3	536368	[jam making set with jars, red coat rack paris...
4	536369	[bath building block word]
...	...	...
19731	581583	[lunch bag red retrospot, 6 chocolate love hea...

19732	581584	[red flock love heart photo frame, 6 chocolate...
19733	581585	[black tea towel classic design, assorted bott...
19734	581586	[large cake stand hanging strawberry, set of 3...
19735	581587	[circus parade lunch box, plasters in tin circ...

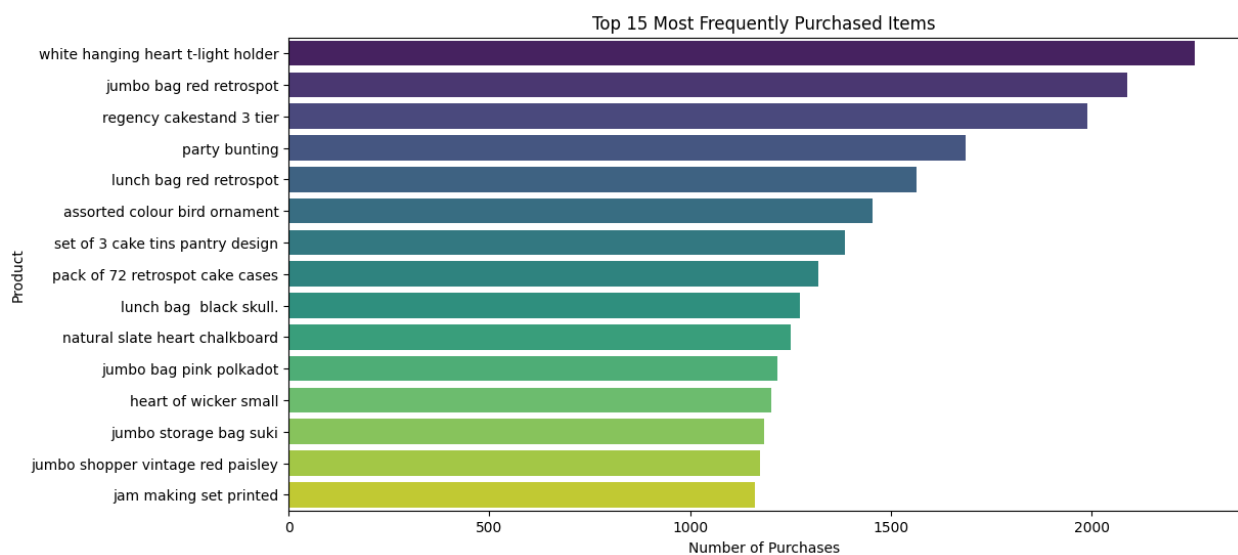
	NumProducts
0	7
1	2
2	11
3	4
4	1
...	...
19731	2
19732	2
19733	21
19734	4
19735	15

[19736 rows x 3 columns]

C:\Users\RayanTech\AppData\Local\Temp\ipykernel_4264\3291989156.py:12: FutureWarning:

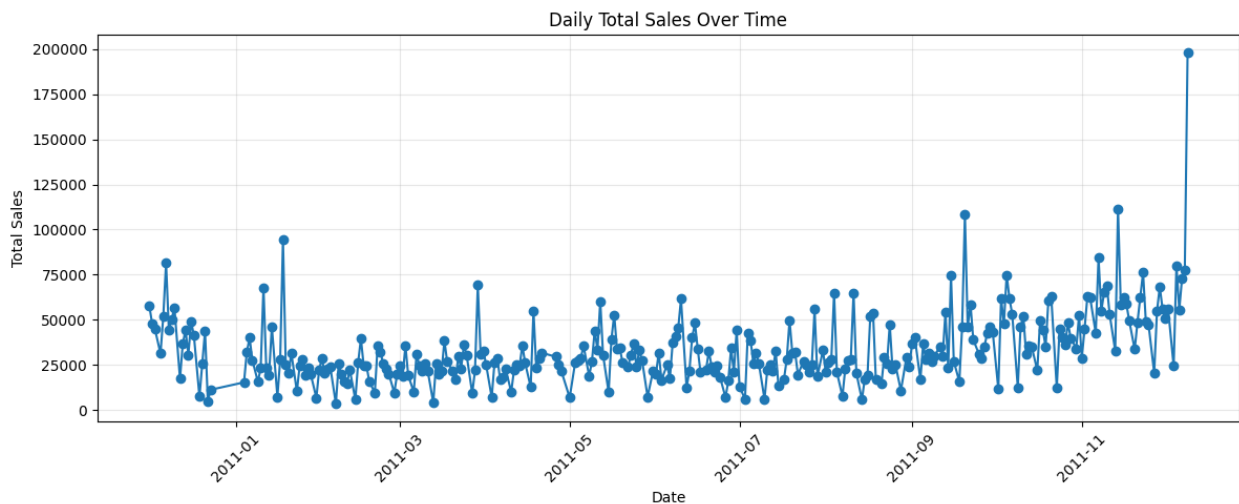
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=top_items.values, y=top_items.index,
palette="viridis")
```



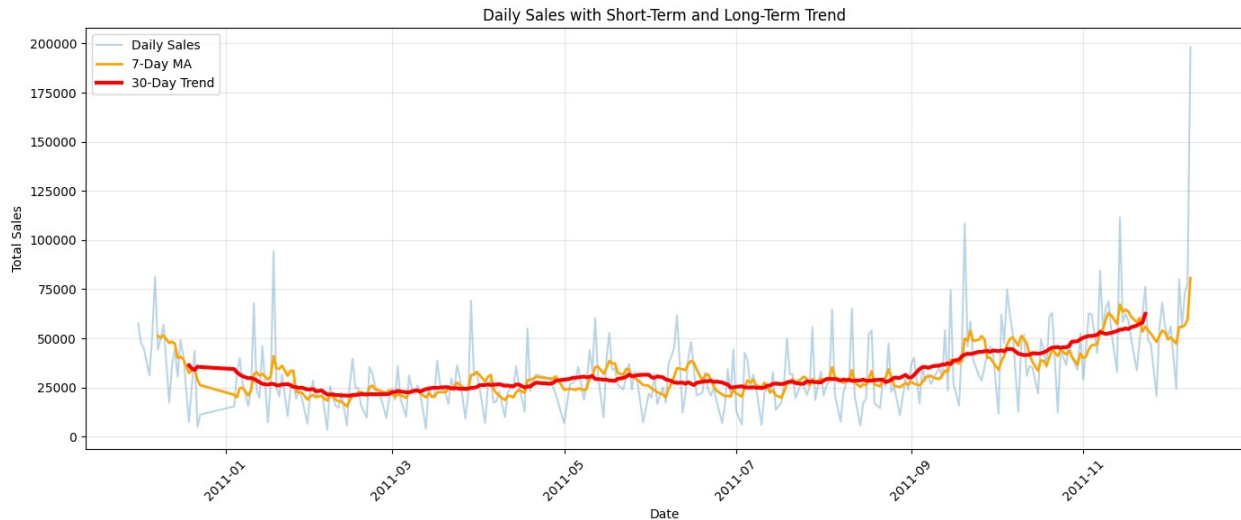
EDA for Time Series

```
plt.figure(figsize=(12,5))
plt.plot(daily_sales["Date"], daily_sales["TotalAmount"], marker='o',
linestyle='-')
plt.xlabel("Date")
plt.ylabel("Total Sales")
plt.title("Daily Total Sales Over Time")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



```
daily_sales["RollingMean_7"] =
daily_sales["TotalAmount"].rolling(window=7).mean()

daily_sales["Trend_30"] =
daily_sales["TotalAmount"].rolling(window=30, center=True).mean()
plt.figure(figsize=(14,6))
plt.plot(daily_sales["Date"], daily_sales["TotalAmount"], alpha=0.3,
label="Daily Sales")
plt.plot(daily_sales["Date"], daily_sales["RollingMean_7"],
color='orange', linewidth=2, label="7-Day MA")
plt.plot(daily_sales["Date"], daily_sales["Trend_30"], color='red',
linewidth=3, label="30-Day Trend")
plt.xlabel("Date")
plt.ylabel("Total Sales")
plt.title("Daily Sales with Short-Term and Long-Term Trend")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



Association Rule Mining

```
transactions = baskets["Description"].apply(
    lambda basket: [str(item) for item in basket]
).tolist()
te = TransactionEncoder()
```

```
te_array = te.fit(transactions).transform(transactions)
df_hot = pd.DataFrame(te_array, columns=te.columns_)
df_hot
```

	10 colour spaceboy pen	12 coloured party balloons \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19731	False	False
19732	False	False
19733	False	False
19734	False	False
19735	False	False

	12 daisy pegs in wood box	12 egg house painted wood \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19731	False	False
19732	False	False
19733	False	False

19734	False	False
19735	False	False
12 ivory rose peg place settings 12 message cards with envelopes \		
0	False	
False		
1	False	
False		
2	False	
False		
3	False	
False		
4	False	
False		
...	...	.
..		
19731	False	
False		
19732	False	
False		
19733	False	
False		
19734	False	
False		
19735	False	
False		
12 pencil small tube woodland 12 pencils small tube red retrospot \		
0	False	
False		
1	False	
False		
2	False	
False		
3	False	
False		
4	False	
False		
...	...	
...		
19731	False	
False		
19732	False	
False		
19733	False	
False		
19734	False	

False
19735
False

False

	12 pencils small tube skull	12 pencils tall tube posy	...	\
0	False	False	...	
1	False	False	...	
2	False	False	...	
3	False	False	...	
4	False	False	...	
...	...	...	...	
19731	False	False	...	
19732	False	False	...	
19733	False	False	...	
19734	False	False	...	
19735	False	False	...	

	zinc herb garden container	zinc metal heart decoration	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	
...	...	...	
19731	False	False	
19732	False	False	
19733	False	False	
19734	False	False	
19735	False	False	

	zinc sweetheart soap dish	zinc sweetheart wire letter rack	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	
...	...	...	
19731	False	False	
19732	False	False	
19733	False	False	
19734	False	False	
19735	False	False	

	zinc t-light holder star large	zinc t-light holder stars small
\		
0	False	False
1	False	False
2	False	False

3	False	False
4	False	False
...	...	...
19731	False	False
19732	False	False
19733	True	False
19734	False	False
19735	False	False
zinc top 2 door wooden shelf zinc willie winkie candle stick		
\		
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19731	False	False
19732	False	False
19733	False	True
19734	False	False
19735	False	False
zinc wire kitchen organiser zinc wire sweetheart letter tray		
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
19731	False	False

19732	False	False
19733	False	False
19734	False	False
19735	False	False

[19736 rows x 3196 columns]

```
frequent_itemsets = apriori(df_hot, min_support=0.01,  
use_colnames=True)  
rules = association_rules(frequent_itemsets, metric="support",  
min_threshold=0)  
print(len(rules))
```

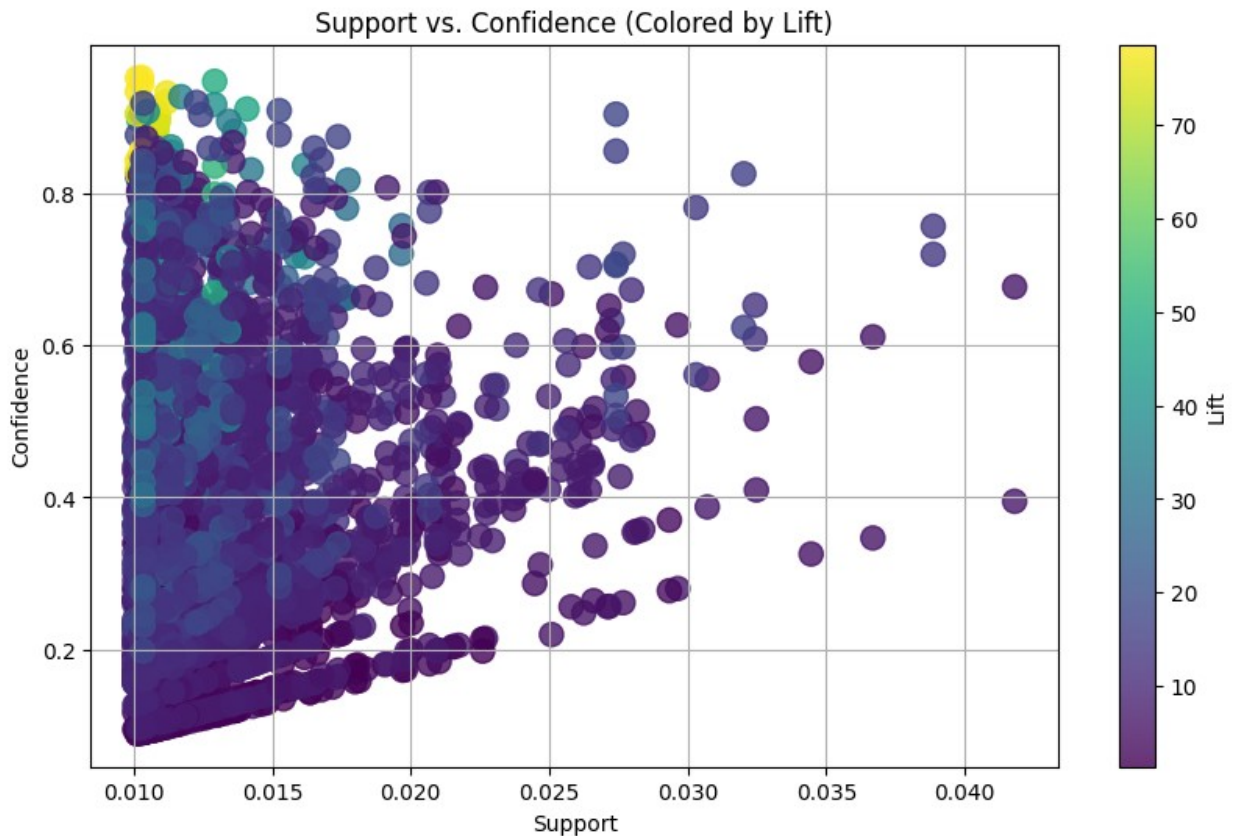
3208

```
plt.figure(figsize=(10,6))
```

```
scatter = plt.scatter(  
    rules["support"],  
    rules["confidence"],  
    c=rules["lift"],  
    s=120,  
    alpha=0.8,  
)
```

```
plt.colorbar(scatter, label="Lift")  
plt.xlabel("Support")  
plt.ylabel("Confidence")  
plt.title("Support vs. Confidence (Colored by Lift)")  
plt.grid(True)
```

```
plt.show()
```



```
frequent_items = frequent_itemsets.sort_values("support",
ascending=False)
frequent_items
```

	support	itemsets
789	0.114309	(white hanging heart t-light holder)
355	0.105847	(jumbo bag red retrospot)
566	0.100730	(regency cakestand 3 tier)
473	0.085377	(party bunting)
395	0.079246	(lunch bag red retrospot)
...	...	...
825	0.010032	(6 ribbons rustic charm, natural slate heart c...
1774	0.010032	(jumbo bag toys, jumbo shopper vintage red pai...
1064	0.010032	(jam making set printed, recycling bag retrospot)
1062	0.010032	(jam making set printed, party bunting)
1779	0.010032	(lunch bag alphabet design, lunch bag suki des...

```
[1874 rows x 2 columns]
```

```
rules = association_rules(frequent_items, metric="support",
min_threshold=0.01)
rules = rules.sort_values("support", ascending=False)
rules["support"]
len(rules)
```

3208

support: 0.1 --> 0 rows 0.09 --> 0 rows 0.08 --> 0 rows 0.07 --> 0 rows 0.06 --> 0 rows 0.05 --> 0 rows 0.04 --> 2 rows 0.03 --> 18 rows* 0.02 --> 180 rows 0.01 --> 608 rows I decided to keep the support of 0.03 because it was the largest support in which there were rules with lifts larger than 1; I tried with larger supports, however, none of them had a lift larger than 1 meaning they were frequent but not strong like they were purchased a lot together but more separately. 0.06, 0.05 --> no lift larger than 1 0.04 --> 2 ones with lift larger than 1

```
rules = association_rules(frequent_items, metric="lift",
min_threshold=1)
rules = rules.sort_values("lift", ascending=False)
rules["lift"]
len(rules)
```

3208

```
rules = association_rules(frequent_items, metric="confidence",
min_threshold=0.9)
rules = rules.sort_values("confidence", ascending=False)
rules["confidence"]
len(rules)
```

20

confidence: 0.5 187* 0.6 98 0.7 46 0.8 17 0.9 2 considering that the lift and support thresholds are already picked, I chose the largest confidence that provided me with at least 10 rules. Therefore I will have 10 strongest rules.

```
rules['Imbalance_Ratio'] = (
    (rules['antecedent support'] - rules['consequent support']).abs()
/
    (rules['antecedent support'] + rules['consequent support'] -
rules['support'])
)
rules['Imbalance_Ratio'] = rules['Imbalance_Ratio'].round(4)
print(len(rules))
rules.head()
```

20

	antecedents \	
13	(herb marker parsley, herb marker thyme)	
18	(herb marker thyme, herb marker basil)	
3	(regency tea plate roses, regency tea plate pink)	
14	(herb marker parsley, herb marker rosemary)	
19	(herb marker rosemary, herb marker basil)	
	consequents	antecedent support consequent support
\		

13	(herb marker rosemary)	0.010792	0.012161
18	(herb marker rosemary)	0.010640	0.012161
3	(regency tea plate green)	0.013630	0.019305
14	(herb marker thyme)	0.010894	0.012009
19	(herb marker thyme)	0.010843	0.012009

	support	confidence	lift	representativity	leverage
conviction \					
13	0.010286	0.953052	78.372613	1.0	0.010155
21.040981					
18	0.010134	0.952381	78.317460	1.0	0.010004
20.744629					
3	0.012921	0.947955	49.104587	1.0	0.012657
18.843357					
14	0.010286	0.944186	78.626396	1.0	0.010155
17.701514					
19	0.010134	0.934579	77.826413	1.0	0.010004
15.102156					

	zhangs_metric	jaccard	certainty	kulczynski	Imbalance_Ratio
13	0.998011	0.812000	0.952474	0.899442	0.1080
18	0.997849	0.800000	0.951795	0.892857	0.1200
3	0.993172	0.645570	0.946931	0.808623	0.2835
14	0.998155	0.815261	0.943508	0.900363	0.0884
19	0.997972	0.796813	0.933784	0.889231	0.0916

```

filtered_rules = rules[
    (rules["lift"] > 1) &
    (rules["support"] > 0.01) &
    (rules["confidence"] >= 0.9) &
    (rules["Imbalance_Ratio"] <= 0.5) &
    (rules["kulczynski"] >= 0.5)
]

filtered_rules = filtered_rules.sort_values("lift", ascending=False)
print(len(filtered_rules))
top10_rules = filtered_rules.head(10)
top10_rules

```

15

	antecedents \
14	(herb marker parsley, herb marker rosemary)
13	(herb marker parsley, herb marker thyme)
18	(herb marker thyme, herb marker basil)
19	(herb marker rosemary, herb marker basil)

```

8          (herb marker thyme)
9          (herb marker rosemary)
15         (herb marker thyme, herb marker rosemary)
10         (herb marker parsley)
17         (herb marker thyme, herb marker rosemary)
3  (regency tea plate roses, regency tea plate pink)

```

	consequents	antecedent support	consequent support
\			
14	(herb marker thyme)	0.010894	0.012009
13	(herb marker rosemary)	0.010792	0.012161
18	(herb marker rosemary)	0.010640	0.012161
19	(herb marker thyme)	0.010843	0.012009
8	(herb marker rosemary)	0.012009	0.012161
9	(herb marker thyme)	0.012161	0.012009
15	(herb marker parsley)	0.011198	0.012059
10	(herb marker rosemary)	0.012059	0.012161
17	(herb marker basil)	0.011198	0.012262
3	(regency tea plate green)	0.013630	0.019305

	support	confidence	lift	representativity	leverage
conviction \					
14	0.010286	0.944186	78.626396	1.0	0.010155
17.701514					
13	0.010286	0.953052	78.372613	1.0	0.010155
21.040981					
18	0.010134	0.952381	78.317460	1.0	0.010004
20.744629					
19	0.010134	0.934579	77.826413	1.0	0.010004
15.102156					
8	0.011198	0.932489	76.681716	1.0	0.011052
14.632372					
9	0.011198	0.920833	76.681716	1.0	0.011052
12.479892					
15	0.010286	0.918552	76.170349	1.0	0.010151
12.129718					
10	0.010894	0.903361	74.286415	1.0	0.010747
10.221991					
17	0.010134	0.904977	73.804271	1.0	0.009996
10.394768					

```
3    0.012921    0.947955  49.104587                1.0  0.012657
18.843357
```

	zhangs_metric	jaccard	certainty	kulczynski	Imbalance_Ratio
14	0.998155	0.815261	0.943508	0.900363	0.0884
13	0.998011	0.812000	0.952474	0.899442	0.1080
18	0.997849	0.800000	0.951795	0.892857	0.1200
19	0.997972	0.796813	0.933784	0.889231	0.0916
8	0.998955	0.863281	0.931658	0.926661	0.0117
9	0.999109	0.863281	0.919871	0.926661	0.0117
15	0.998047	0.792969	0.917558	0.885747	0.0664
10	0.998581	0.817490	0.902172	0.899597	0.0076
17	0.997622	0.760456	0.903798	0.865712	0.0798
3	0.993172	0.645570	0.946931	0.808623	0.2835

```
top10_rules[[
    "antecedents",
    "consequents",
    "support",
    "confidence",
    "lift"
]]
```

	antecedents \
14	(herb marker parsley, herb marker rosemary)
13	(herb marker parsley, herb marker thyme)
18	(herb marker thyme, herb marker basil)
19	(herb marker rosemary, herb marker basil)
8	(herb marker thyme)
9	(herb marker rosemary)
15	(herb marker thyme, herb marker rosemary)
10	(herb marker parsley)
17	(herb marker thyme, herb marker rosemary)
3	(regency tea plate roses, regency tea plate pink)

	consequents	support	confidence	lift
14	(herb marker thyme)	0.010286	0.944186	78.626396
13	(herb marker rosemary)	0.010286	0.953052	78.372613
18	(herb marker rosemary)	0.010134	0.952381	78.317460
19	(herb marker thyme)	0.010134	0.934579	77.826413
8	(herb marker rosemary)	0.011198	0.932489	76.681716
9	(herb marker thyme)	0.011198	0.920833	76.681716
15	(herb marker parsley)	0.010286	0.918552	76.170349
10	(herb marker rosemary)	0.010894	0.903361	74.286415
17	(herb marker basil)	0.010134	0.904977	73.804271
3	(regency tea plate green)	0.012921	0.947955	49.104587

```
plt.figure(figsize=(10,6))
```

```
scatter = plt.scatter(
```

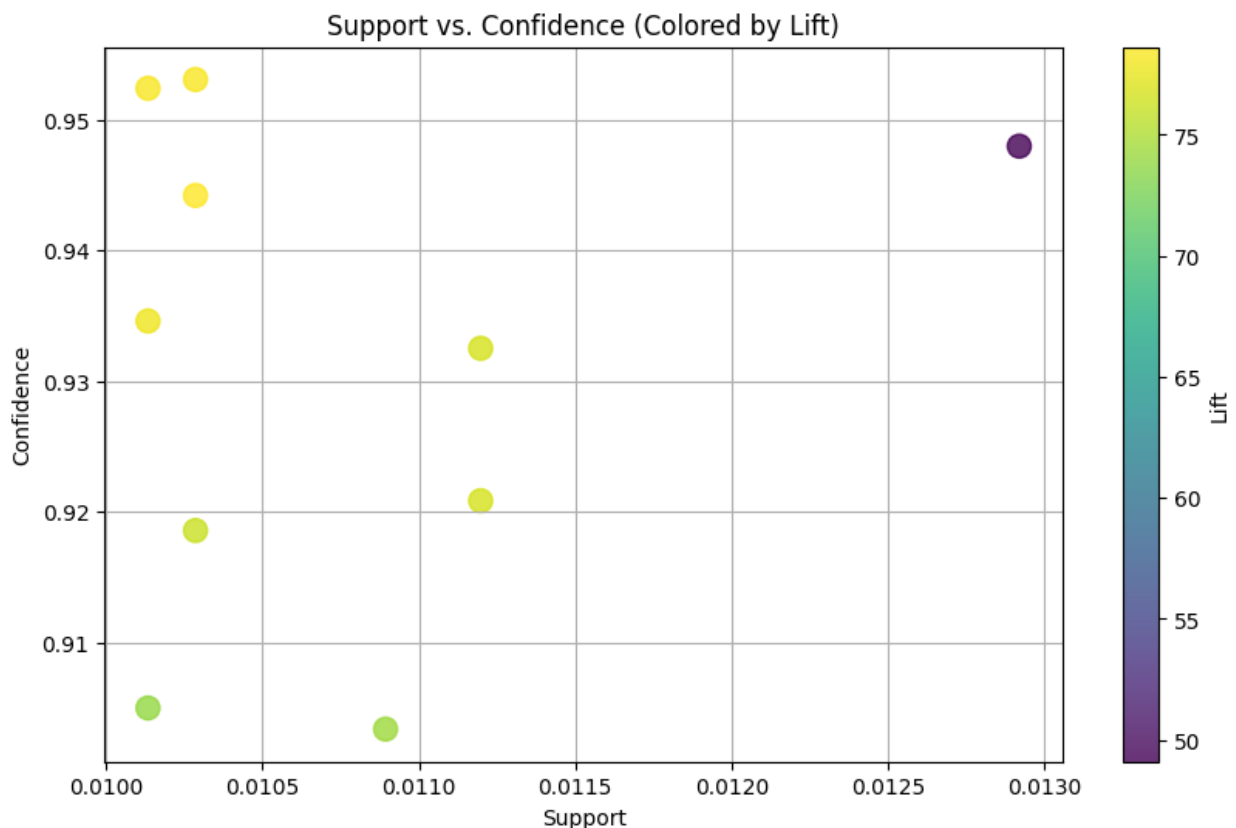
```

top10_rules["support"],
top10_rules["confidence"],
c=top10_rules["lift"],
s=120,
alpha=0.8,
)

plt.colorbar(scatter, label="Lift")
plt.xlabel("Support")
plt.ylabel("Confidence")
plt.title("Support vs. Confidence (Colored by Lift)")
plt.grid(True)

plt.show()

```



The biggest business value is brought by the most interesting rules meaning the ones with highest lift After filtering the lift, the confidence is prioritized meaning how likeble it is for the item Y to be purchased if item X is purchased And after that the support is prioritized meaning how big the opportunity is so I chose this formula to assign some weight to each of the factors in the right order: $\text{top10_rules}["\text{biz_score}"] = (\text{top10_rules}["\text{lift}"] * 0.5 + \text{top10_rules}["\text{confidence}"] * 0.3 + \text{top10_rules}["\text{support}"] * 0.2)$

```

top10_rules["biz_score"] = (
    top10_rules["lift"] * 0.5 +

```

```

    top10_rules["confidence"] * 0.3 +
    top10_rules["support"] * 0.2
)

best5 = top10_rules.sort_values("biz_score", ascending=False).head(5)

plt.figure(figsize=(10,6))

scatter = plt.scatter(
    best5["support"],
    best5["confidence"],
    c=best5["lift"],
    s=120,
    alpha=0.8,
)

plt.colorbar(scatter, label="Lift")
plt.xlabel("Support")
plt.ylabel("Confidence")
plt.title("Support vs. Confidence (Colored by Lift)")
plt.grid(True)

plt.show()

print(
    best5[[
        "antecedents",
        "consequents",
        "support",
        "confidence",
        "lift"
    ]]
)

```

C:\Users\RayanTech\AppData\Local\Temp\ipykernel_4264\4282307443.py:1:

SettingWithCopyWarning:

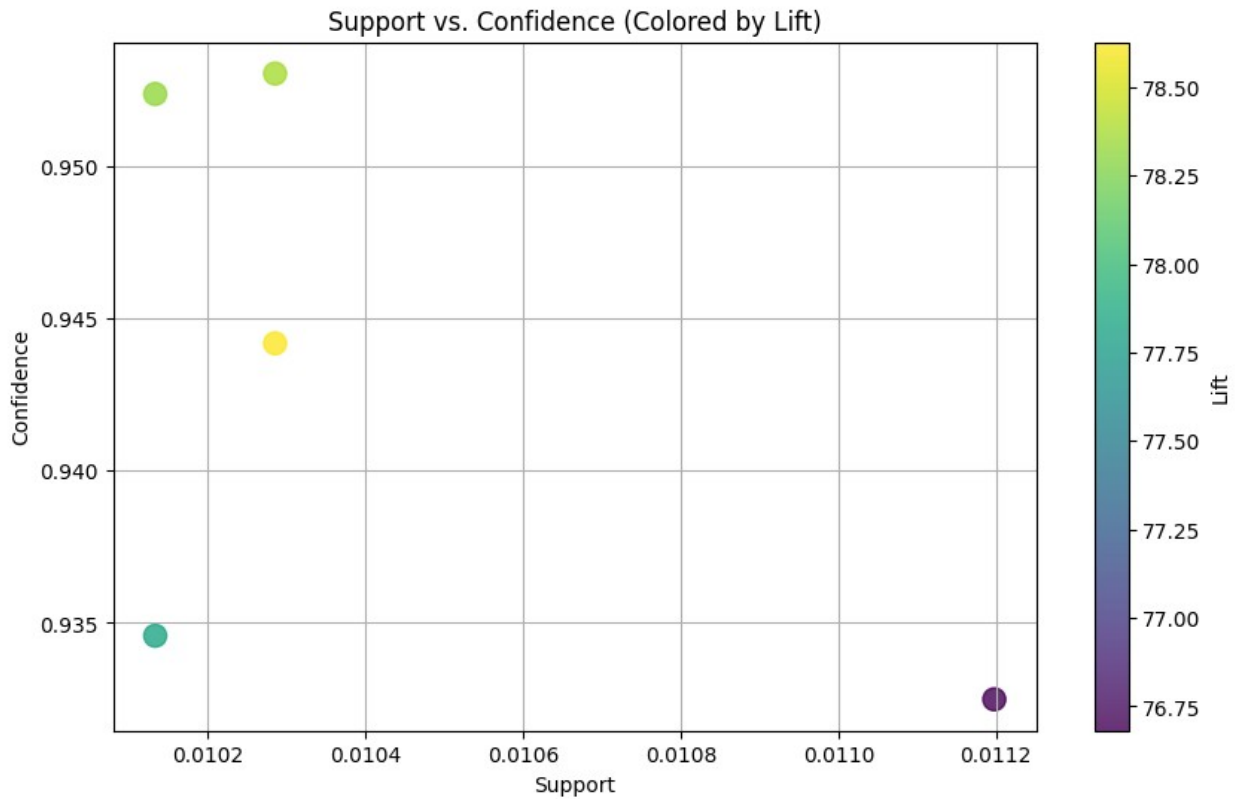
A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
top10_rules["biz_score"] = (
```

	consequents \	antecedents
14	(herb marker parsley, herb marker rosemary)	(herb marker thyme)
13	(herb marker parsley, herb marker thyme)	(herb marker rosemary)
18	(herb marker thyme, herb marker basil)	(herb marker rosemary)
19	(herb marker rosemary, herb marker basil)	(herb marker thyme)
8		(herb marker thyme) (herb marker rosemary)

	support	confidence	lift
14	0.010286	0.944186	78.626396
13	0.010286	0.953052	78.372613
18	0.010134	0.952381	78.317460
19	0.010134	0.934579	77.826413
8	0.011198	0.932489	76.681716

1. Bundle Promotions (Cross-sell bundles)

Rules with high confidence + high lift identify products that customers consistently buy together.

How to use the rules:

Create special bundles (e.g., "Buy X, get Y at 20% off").

Offer automated recommendations on checkout pages ("Customers who bought X also bought Y").

Use email campaigns: "You purchased X — here's a discount on Y."

Business impact:

Increases average order value (AOV).

Improves conversion rate through targeted suggestions.

1. Store/Product Placement

Rules with good support + high lift show which items should be located close to each other.

How to use the rules:

Place items that frequently co-occur in the same aisle or shelf.

In e-commerce, place them in "Recommended for you" or "Frequently bought together" sections.

Use them to optimize warehouse picking routes to reduce picking time.

Business impact:

Reduces shopping friction.

Increases impulse purchases.

Improves online recommendation accuracy.

1. Targeted Marketing Campaigns

Rules with high lift but lower support reveal small-but-valuable niche customer patterns.

How to use the rules:

Segment customers based on their previous purchases.

Send personalized offers: "Since you bought X, here's 10% off Y — customers like you often buy both."

Optimize remarketing ads for products that frequently co-occur.

Business impact:

Raises ROI of ads and email campaigns.

Improves customer retention through personalization.

Part 2: Time Series Forecasting

```

daily_sales["Date"] = pd.to_datetime(daily_sales["Date"])
ts = daily_sales.set_index("Date")["TotalAmount"]

split_idx = int(len(ts) * 0.8)
train = ts.iloc[:split_idx]
test = ts.iloc[split_idx:]

window_size = 7
forecast_days = 14

ts_sma = train.rolling(window=window_size).mean()

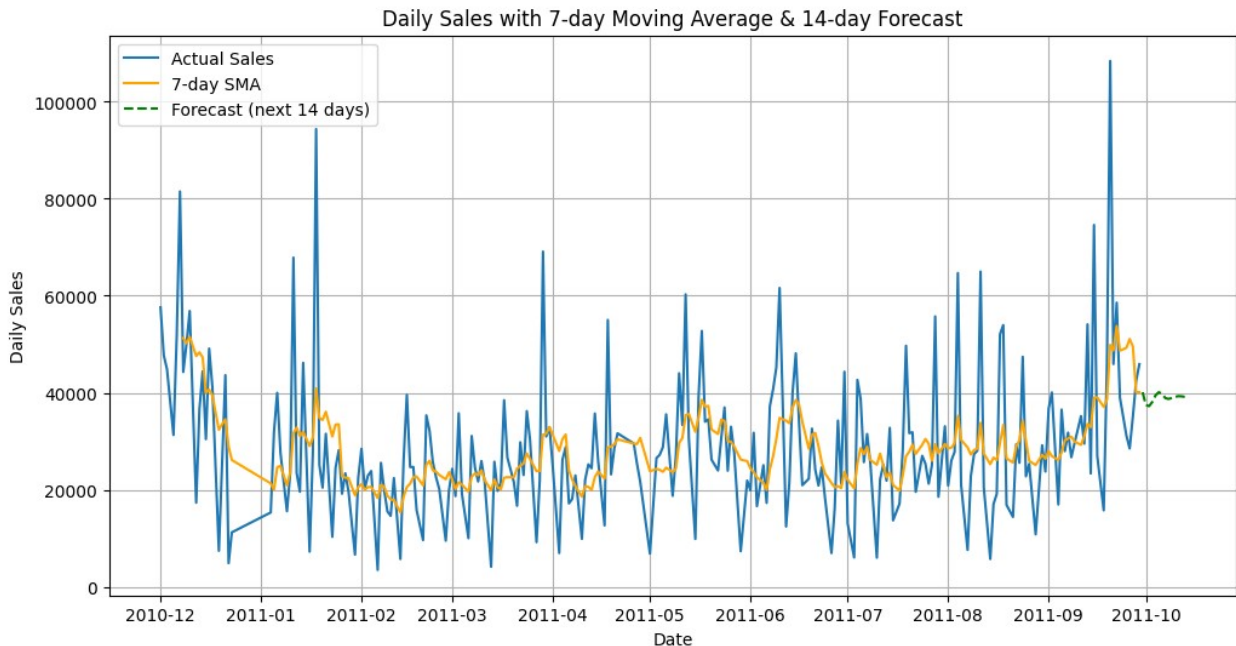
last_window = train[-window_size:].tolist()
forecast_values = []

for i in range(forecast_days):
    next_val = sum(last_window[-window_size:]) / window_size
    forecast_values.append(next_val)
    last_window.append(next_val)

last_date = train.index[-1]
forecast_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
                                periods=forecast_days)

plt.figure(figsize=(12,6))
plt.plot(train, label="Actual Sales")
plt.plot(ts_sma, label=f"{window_size}-day SMA", color="orange")
plt.plot(forecast_dates, forecast_values, label="Forecast (next 14 days)", linestyle='--', color='green')
plt.xlabel("Date")
plt.ylabel("Daily Sales")
plt.title("Daily Sales with 7-day Moving Average & 14-day Forecast")
plt.legend()
plt.grid(True)
plt.show()

```



There are 2 cases of prediction generations using SMA method; at each timestamp there are 2 possibilities: adding the predicted value to the window or replacing the actual value. The point is that MA is a smoothing method and after 2 iterations of window size it completely flattens. However, replacing the actual values avoids that kind of flattening and improves accuracy a little bit. Below comes both cases and as can be seen in the case of no replacement, the method made more realistic predictions.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

daily_sales["Date"] = pd.to_datetime(daily_sales["Date"])
ts = daily_sales.set_index("Date")["TotalAmount"].sort_index()

split_idx = int(len(ts) * 0.8)
train = ts.iloc[:split_idx]
test = ts.iloc[split_idx:]

window_size = 7
forecast_days = len(test)
last_window = list(train.iloc[-window_size:])

forecast_values = []

for i in range(forecast_days):
    next_val = sum(last_window[-window_size:]) / window_size
    forecast_values.append(next_val)
    last_window.append(next_val)
```

```

forecast = pd.Series(forecast_values, index=test.index)

mae = mean_absolute_error(test, forecast)
mse = mean_squared_error(test, forecast)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((test - forecast) / test)) * 100

print("MAE :", mae)
print("RMSE:", rmse)
print("MAPE:", mape)

plt.figure(figsize=(12,8))

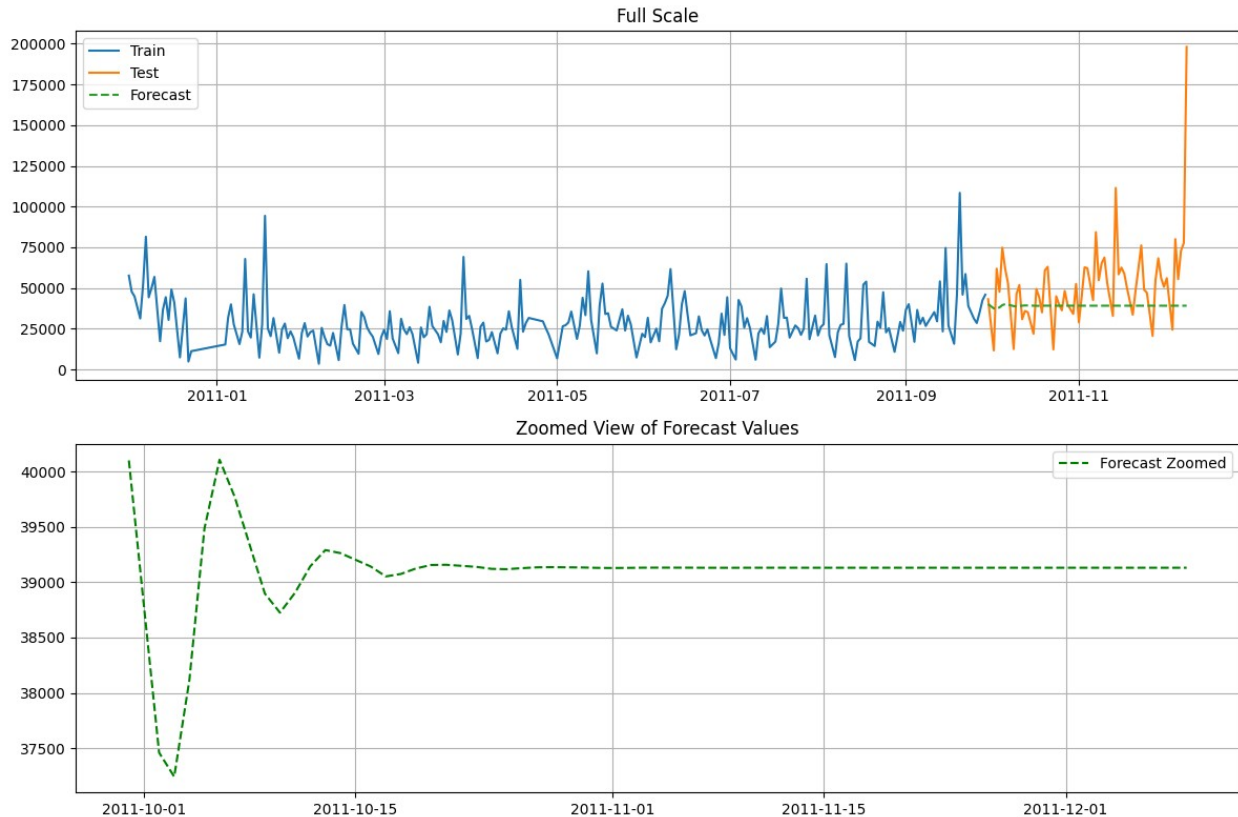
plt.subplot(2,1,1)
plt.plot(train, label="Train")
plt.plot(test, label="Test")
plt.plot(forecast, label="Forecast", linestyle='--')
plt.title("Full Scale")
plt.legend()
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(forecast, label="Forecast Zoomed", linestyle='--',
color='green')
plt.title("Zoomed View of Forecast Values")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

MAE : 19607.19062258237
RMSE: 29521.287093992727
MAPE: 40.1250740327558

```



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error

daily_sales["Date"] = pd.to_datetime(daily_sales["Date"])
ts = daily_sales.set_index("Date")["TotalAmount"].sort_index()

split_idx = int(len(ts) * 0.8)
train = ts.iloc[:split_idx]
test = ts.iloc[split_idx:]

window_size = 7

rolling_window = list(train.iloc[-window_size:])

forecast_values = []

for actual in test:
    forecast = sum(rolling_window[-window_size:]) / window_size
    forecast_values.append(forecast)
    rolling_window.append(actual)

forecast = pd.Series(forecast_values, index=test.index)
```

```

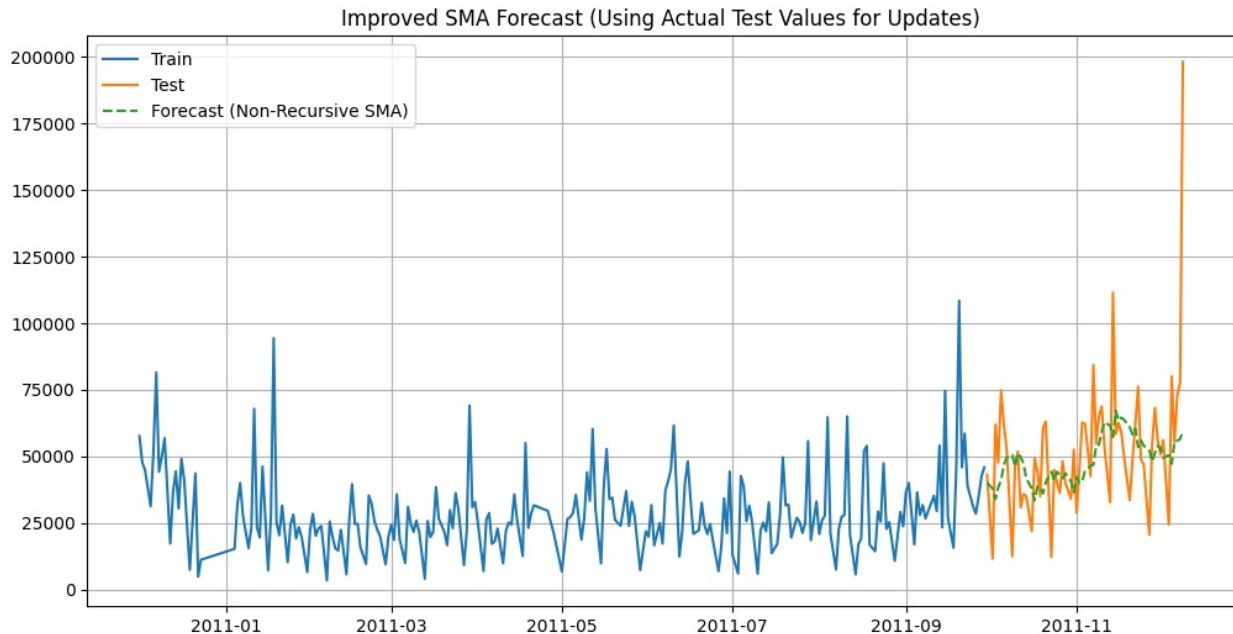
mae = mean_absolute_error(test, forecast)
mse = mean_squared_error(test, forecast)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((test - forecast) / test)) * 100

print("MAE :", mae)
print("RMSE:", rmse)
print("MAPE:", mape)

# Plot
plt.figure(figsize=(12,6))
plt.plot(train, label="Train")
plt.plot(test, label="Test")
plt.plot(forecast, label="Forecast (Non-Recursive SMA)",
linestyle='--')
plt.title("Improved SMA Forecast (Using Actual Test Values for
Updates)")
plt.legend()
plt.grid(True)
plt.show()

MAE : 16107.386674473066
RMSE: 25402.904763274044
MAPE: 39.73200669653199

```



Analysis and Interpretation

SMA is one of the least reliable forecasting methods for anything beyond very short-term prediction and the flattening we observed is exactly why. Reason 1: SMA cannot model trend

If data is trending upward or downward: SMA smooths it out, forecast stays “in the middle” and completely misses the trend

Reason 2: SMA cannot model seasonality If sales have weekly patterns (Mon–Sun cycle), SMA ignores this.

Reason 3: Recursive SMA always collapses to a constant Recursive forecasting means: Predict tomorrow using today's forecast, Predict next day using today's predicted value

Evaluation Metrics: MAE = $(1/n) \sum_{i=1}^n |y_i - \hat{y}_i|$ mean squared error (MSE), the average squared difference between the value observed in a statistical study and the values predicted from a model. It provides an estimate of the size of the inaccuracy, but not its direction (e.g., over or under-prediction). Resiliency to outliers. The MAE is not as impacted by extreme results as other metrics, such as Mean Squared Error (MSE), are. This makes it an appropriate measure for datasets that include outliers or extreme values.

RMSE = $\sqrt{[(\sum (P_i - O_i)^2) / n]}$ It is one of the standard ways to measure the error rate of a model in predicting quantitative data. It is a metric predominantly utilized in regression analysis and forecasting, where accuracy matters significantly. The lower the RMSE, the better the model's ability to predict accurately. Conversely, a higher RMSE signifies a greater discrepancy between the predicted and actual outcomes.

MAPE Mean absolute percentage error measures the average magnitude of error produced by a model, or how far off predictions are on average. A MAPE value of 20% means that the average absolute percentage difference between the predictions and the actuals is 20% $MAPE = n^{-1} \sum (|Actual_i - Forecast_i| / Actual_i) \times 100$